

Project 2 - Hidden Markov Models – Artificial Intelligence

Teacher: Stephan Schiffel

March 3, 2026

Use the Piazza page or Discord if you have any questions or problems with the assignment. **Start early**, so you still have time to ask in case of problems!

This assignment is supposed to be done in a group of 2-3 students. Note that everyone in the group needs to understand the whole solution even if you distribute the work.

1 Time Estimate

20 hours per student in addition to the time spend in the labs, assuming you work in groups of 2-3 students, did the previous lab assignments and attended the lectures on Bayesian Networks and DBNs.

2 Overview

In this project you will build a Hidden Markov Model (HMM)–based phoneme recognizer from speech audio. You will use the CMU ARCTIC speech corpus (speaker: US English slt female) and implement:

- Supervised parameter learning from frame-level phonetic labels,
- HMM inference: forward algorithm (filtering) and Viterbi decoding,

- (Optional) EM / Baum–Welch refinement.

The raw data preparation (dataset download, train/validation/test split, and feature extraction) is already done for you. You will work directly with the precomputed feature files.

You will evaluate how well your HMM predicts phoneme sequences from the speech signal.

3 Context

Speech recognition is the process of converting acoustic speech signals into a sequence of linguistic units (words, phonemes, or subword tokens). At a high level, modern automatic speech recognition (ASR) systems combine three components:

- Acoustic front-end: transforms raw audio into time-series feature vectors (e.g., MFCCs, filterbanks, or learned embeddings) that summarize short-time spectral and temporal patterns.
- Acoustic model: probabilistically relates feature vectors to discrete linguistic states (typically phonemes, context-dependent phones, or subword units). Classic approaches use generative models (HMMs with Gaussian or GMM emissions); modern systems often use discriminative models (neural networks) to estimate $P(\text{state} \mid \text{features})$.
- Language model and decoder: constrains and scores sequences of linguistic units to form valid words and phrases, using transition models and statistical/learned language priors; a decoder (Viterbi, beam search, WFST) finds the best overall transcription given acoustic and language scores.

This project focuses on the acoustic-modeling and decoding stages:

- Data preparation: align feature frames with phonetic labels to create supervised training data.

- Supervised estimation: estimate HMM parameters (initial distribution, transition matrix, state emission parameters).
- Inference: implement forward filtering (online posteriors) and Viterbi decoding (most likely state sequence).
- Discriminative options: optionally replace Gaussian emissions with classifiers (e.g., random forest, NN).

4 Provided Material

The data set consists of 1132 audio recordings (utterances) of english sentences being spoken by one speaker (.wav files). For each recording there is a label file with a sequence of time-stamped phonemes that are being uttered (.lab files). There are 40 phonemes using the ARPAbet codes plus pau for "pause".

You are given:

- The CMU ARCTIC dataset,
- A script `train_test_split.py`, that randomly splits the recordings into three sets: train, test and validation, and creates 3 files `train.txt`, `val.txt`, `test.txt` containing the names of utterances in each set.
- A script `generate_features.py`, that produces a feature vector for each 10ms window of each recording,
- The precomputed feature files (one .npy file per utterance).

4.1 How to Use the Provided Data

When building and evaluating your HMM, you must:

- Use the utterances listed in `train.txt` to learn model parameters,
- Use `val.txt` (optional) to tune hyperparameters or check for overfitting,

- Use `test.txt` strictly for final evaluation of the created model.

In your code, you will typically read these lists of basenames and then load the corresponding feature files and label files for each split.

4.2 Features

The script `generate_features.py`:

- Reads each `.wav` file and:
 - Frames it into 25 ms wide windows with 10 ms shift, (meaning windows overlap)
 - Computes 13 MFCC coefficients per frame,
 - Computes first-order deltas (Δ) and second-order deltas ($\Delta\Delta$),
 - Concatenates them into 39-dimensional feature vectors per frame.
- Saves the resulting feature matrix as a NumPy `.npy` file with the same basename as the `.wav` file, e.g., for input `arctic_a0001.wav`, it creates `arctic_a0001.npy`.
- Each `.npy` file contains a 2D array of shape `(num_frames, 39)`. Where `num_frames` depends on the length of the recording.

In your HMM code you will:

- Load the feature matrices (e.g. using `numpy.load("arctic_a0001.npy")`) as the observation sequences $O_{1:T}$,
- Align each frame to a phoneme label (from the corresponding `.lab` file) to obtain the state sequence $X_{1:T}$ for supervised training,
- Use these feature matrices throughout training, inference, and evaluation.

You do *not* need to run the scripts or implement feature extraction yourself; you only need to understand the format of their outputs and consume the provided files correctly.

5 HMM Design

5.1 Hidden State Space

Use a phoneme-based state space. That is, each state is one of the phonemes appearing in the dataset. That means, you will have a single discrete state variable X_t with ≈ 40 possible values. To make computations more efficient, the state should be an integer variable: $X_t \in \{0, \dots, K-1\}$. Therefore, you will need to map the phoneme labels (strings) to integers.

5.2 Observation Model

Observations are the precomputed frame-level acoustic feature vectors from the provided `.npy` files.

Thus, each time step t has an observation

$$O_t \in \mathbb{R}^d, \quad d = 39$$

(corresponding to 13 MFCCs + 13 deltas + 13 delta-deltas).

Thus, the sensor model is continuous (and not discrete like in the lecture example). Therefore, use a continuous-emission HMM:

- For each hidden state (phoneme) i , model emissions as a multivariate Gaussian:

$$P(O_t \mid X_t = i) = \mathcal{N}(\mu_i, \Sigma_i).$$

- $\mu_i \in \mathbb{R}^d$ is the mean of the expected sensor output in state i .
- $\Sigma_i \in \mathbb{R}^d \times \mathbb{R}^d$ is the covariance matrix of that distribution for state i .

6 Tasks

6.1 Data Preparation

Loading Feature Sequences.

- Read `train.txt`, `val.txt`, and `test.txt` to obtain basenames for each split.
- For each basename (e.g. `arctic_a0001`):
 - Load the corresponding feature matrix (e.g., `arctic_a0001.npy`),
 - Load the corresponding phonetic transcript (e.g., `arctic_a0001.lab`).

Alignment and Label Mapping.

- Use the phoneme time stamps from the `.lab` file to assign a phoneme label to each frame in the feature matrix,
- Define a phoneme set and a mapping from phoneme symbols to integer indices (including a silence/pause label),
- Apply this mapping to obtain a sequence of integer labels $X_{1:T}$ matching the feature frames $O_{1:T}$.

After this step, for each utterance you should have:

- A sequence of feature vectors $O_{1:T}$ loaded from `.npy`,
- A corresponding sequence of phoneme labels $X_{1:T}$ from the aligned `.lab` with the same length as $O_{1:T}$.

These $X_{1:T}$ and $O_{1:T}$ sequences are the data that you will use in the next step to estimate the parameters of the HMM.

6.2 Supervised Parameter Estimation

Using the aligned training data (utterances listed in `train.txt`) estimate the parameters (π, A, μ, Σ) for the Hidden Markov Model:

Initial Distribution π . Estimate

$$\pi_i = P(X_1 = i)$$

from the first labeled frame of each training utterance.

Transition Matrix A . Estimate

$$a_{ij} = P(X_{t+1} = j \mid X_t = i)$$

by counting transitions between successive labelled frames and normalizing each row.

Emission Parameters (Gaussian distributions $\mathcal{N}(\mu_i, \Sigma_i)$). For each state i :

- Collect all feature vectors assigned to phoneme i in the training set,
- Compute

$$\mu_i = \text{mean of features for state } i,$$

$$\Sigma_i = \text{sample covariance of features for state } i.$$

You should use regularization to avoid the covariance matrices to become singular (e.g., by adding small numbers to the diagonal).

6.3 Inference Algorithms

Implement:

Forward Algorithm (Filtering) Compute

$$\alpha_i(t) = P(O_{1:t}, X_t = i)$$

(or a scaled version for numerical stability), and from this obtain the filtering distribution

$$P(X_t \mid O_{1:t}).$$

Viterbi Algorithm. Compute the most likely state sequence for each test utterance:

$$\hat{X}_{1:T} = \arg \max_{X_{1:T}} P(X_{1:T} \mid O_{1:T}).$$

Use log-probabilities or scaling to avoid numerical underflow.

6.4 Evaluation

Evaluate both algorithms on training and test sets (utterances listed in `train.txt` and `test.txt`), that is:

- For each utterance, predict phoneme sequences with Filtering and Viterbi decoding
- Compare the predicted phoneme sequence with the ground truth label from the aligned label file,
- Report on the overall frame accuracy for both Filtering and Viterbi decoding on both the training and test data.
- For the test set, produce a confusion matrix over phonemes.
- For the test set, compute per-phoneme precision and recall.

For some example utterances, provide the predicted phoneme sequences and compare to the labeled data manually.

6.5 Analysis

Discuss:

- How does the accuracy differ between Filtering and Viterbi? Why? Is the accuracy different between train and test set? Why?
- Which phonemes are most often confused (e.g., similar vowels, stops, silence vs. fricatives)?
- For which phonemes is the model better / worse at predicting them correctly?
- Generally, how well does the approach work?

6.6 Possible Extensions (Optional)

EM / Baum–Welch

- Treat phoneme labels as unknown (for some data) and run Baum–Welch to estimate the model parameters from unlabeled observation sequences.
- You can start from supervised parameters or random initialization.
- Compare performance of Viterbi before and after EM refinement.
- Discuss whether EM helped or hurt and possible reasons.

Improve Model Try to improve the accuracy of the model, e.g., by

- improving the features,
- Hyper-parameter tuning (e.g., window size, hop, regularization parameters for Σ , etc),
- Using 2–3 sub-states per phoneme (left-to-right HMM topology) instead of 1 state,
- replacing the sensor model with more expressive models (e.g., Gaussian Mixture Models),

- replacing the sensor model with Machine Learning models such as Random Forests, kNN, ANN that predict $P(X_t|O_t)$ instead of $P(O_t|X_t)$,
- Adding a simple bigram phoneme language model on top of per-phoneme HMMs,

Describe your changes and evaluate the new model to assess the impact of your changes.

Compare with Other Approaches For example, you could implement a basic frame-wise classifier (e.g., logistic regression, ANN, Random Forest) that ignores sequence structure and compare the performance (e.g., accuracy) to the HMM model.

7 Deliverables

7.1 Code

Provide well-documented and organized code (Python scripts or Jupyter notebooks) for

- Loading and preparing the data,
- HMM training (supervised, optionally EM),
- Inference (Filtering, Viterbi),
- Evaluation and result visualization.

Provide a README text file explaining how to run your code and what the prerequisites are.

7.2 Report

Submit a report (PDF) of approximately 4–6 pages including:

Model Description.

- Exact definition of the state space (phoneme set) and observation model,
- Transition, emission, and initial distributions.

Implementation Details.

- Describe and justify all choices you made when implementing the data preparation or algorithms other than what was defined in this assignment description.
- Describe any optional parts you implemented and improvements you tried to make.
- Describe how you achieved numerical stability of the algorithms and how you tested it.

Results. Provide the results obtained in the evaluation including accuracy, confusion matrix, and illustrative examples.

Discussion. Discussion of the results (see Section 6.5).

8 Grading

- 15% - Correct data preparation
- 15% - Supervised parameter estimation
- 20% - Inference implementations: Filtering and Viterbi
- 15% - Evaluation & plotting
- 10% - Analysis and discussion of results
- 15% - Report quality and completeness

- 10% - Code quality
- Up to 20 bonus points for optional parts